Assignment (Term Paper)

Statistical Programming, WS Term 2025/2026

March 20, 2026

Team Members:

Niloufar Nabi, 7968616

Nargizakhon Khikmatullaeva, 7959951

Zeinab Afrasiabi, 7960208

Assignment (Term Paper)

Section 1

Introduction

Section one is assigned to different tasks such as data loading, data cleaning, handling missing values, grouping and sorting data. In the end, the processed data are visualized using different types of plots and charts, ready to be analyzed and interpreted. **The goal of this section is to prepare the data for analysis and visualization**, ensuring that they are clean, organized, and ready for further exploration.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

url_listing_berlin = (
    "https://raw.githubusercontent.com/Nill2nn/Statistical-Programming/"
    "refs/heads/Nill2nn-patch-excel/listings_berlin.csv"
)
listings_berlin = pd.read_csv(url_listing_berlin)
```

Data Loading and Detecting Missing Values

To start with, the data needed to be loaded into the environment. This involves reading the data from a source, such as a CSV file. We first loaded our CSV file into our public GitHub repository and used the relative URL so everyone could access it. Then, with the .shape method, we checked the dimensions of the data. We explicitly used the following two lines of code:

```
pd.set_option('display.max_columns', None)
pd.set_option('display.width', None)
```

To make sure that all columns are visible when printing the data. The second step is detecting missing values. We used the .isnull() method to identify any missing values in the dataset. This method returns a boolean DataFrame indicating the presence of missing values. Afterward, we collected the missing values and converted them into percentages, presenting them in a table to gain a better understanding of them.

Identifying the highest rate of missing values

Based on the table created, the columns with the highest rate of missing values were 'price', 'host_response_rate', 'review_scores_rating', and 'review_scores_cleanliness'. we will recommend to keep the colums as such: The 'price' column should be kept and imputed because pricing information is crucial for the analysis. Dropping this column would result in a significant loss of valuable insights. Therefore, the missing values in the 'price' column should be imputed using the median price of the listings, which is a common approach for handling missing numerical data.

The 'host_response_rate' column should be removed because it contained a high percentage of missing values and was not considered essential for the analysis. Additionally, this variable is unlikely to play a major role in customers' decision-making.

The missing values in 'review_scores_rating' and 'review_scores_cleanliness' should be kept, as these variables can still provide useful information for customers. Imputing these values should be avoided because the missing values appear random, and replacing them with the mean or median could introduce misleading information " " "

cleaning the data

After identifying the missing values, we proceeded to clean the data. This involved handling the missing values in the dataset. For the 'price' column, we wrote a function to thoroughly clean the string values and convert them into numeric values.

```
def clean_price(price_str):
    if pd.isna(price_str):
        return None
    if isinstance(price_str, str):
        return float(price_str.replace('$', '').replace(',', ''))
    return float(price_str)

listings_berlin['price'] = listings_berlin['price'].apply(clean_price)
```

In this function, we used the .isna() method in Pandas to identify missing values and replace them with NaN. Next, we cleaned the remaining data by converting the values from strings to numeric values, removing any special characters, and converting the data types to the appropriate formats. Finally, we applied the function to the 'price' column and used the .describe() method to check the summary statistics of the cleaned data. below you can seethe result of our cleaning process for the 'price' column which consists of Avarage(mean), maximum, minimum and median(50%).

Statistic	Value
count	9264
mean	201.24
std	1656.99
min	5
25%	70
50%	104
75%	160
max	50000

cleaning bathroom column

The 'bathrooms' column contained a mix of string values with different formats, such as "1 bath", "2 baths", and "half bath". To clean this column, we wrote a function that identify missing values and extracts the numeric values from the strings and converts them into a consistent format. The function also handles cases where the value is "Shared bath" by assigning a specific numeric value (e.g., 0.5) to represent shared bathrooms. in this Function, we preffered ro use .split method to extract the numeric part of the string, which is more efficient and easy to underestand than using regular expressions. this is the result of our cleaning process for the 'bathrooms' column:

the Types of rooms available: [1. , 2.5 , 2. , 1.5 0. , nan , 3. , 0.5 , 4.5 , 8. , 3.5 , 5. , 8.5 4. , 6. , 15.]

converting string values to boolean values

The 'host_is_superhost' and 'instant_bookable' column contained string values of "t" and "f", which represent true and false, respectively. To convert these string values into boolean values, we wrote codes that checks the value of each entry in the 'host_is_superhost' column and returns True for "t" and False for "f". This conversion allows us to work with boolean values in our analysis, making it easier to perform logical operations and comparisons based on whether a host is a superhost or not. then for turning the column 'host_tenure_years' into 'host_since', first we made a copy of our database to avoid any changes to the original data. Then, we created a new column 'host_since' by subtracting the 'host_tenure_years' from the current year (2025). you can see the codes and results below:

```
lb = listings_berlin.copy()
lb['host_tenure_years'] = 2025 - pd.to_datetime(lb['host_since']).dt.year
print(lb['host_tenure_years'].head(5))
```

Host Since	Tenure (Years)
2008-10-19	17
2009-08-25	16
2009-11-18	16
2010-01-08	15
2010-01-18	15

comparing the raw data with log transformed data by visualizing the distribution of the 'price' column

in this task our Goal was to compare the raw data with log-transformed data to understand the distribution of the 'price' column. here you can see the plots:

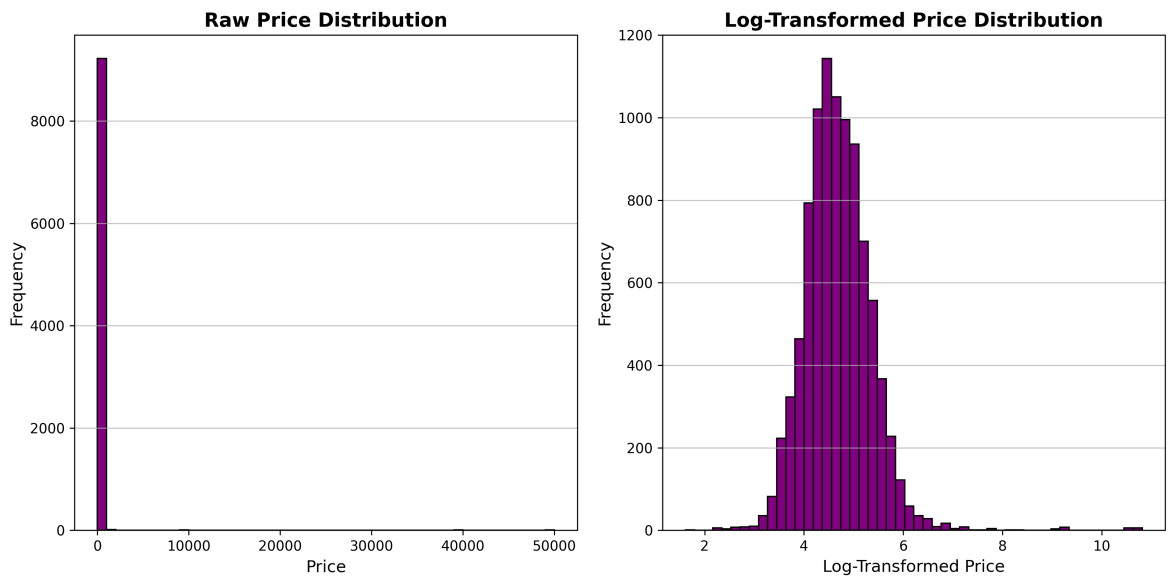

Figure 1: price_distributions.png

as you can see from the plots, the raw price distribution is right-skewed, with a long tail towards higher prices. This indicates that there are a few listings with very high prices compared to the majority of listings. On the other hand, the log-transformed price distribution appears more symmetric and closer to a normal distribution. This transformation helps to reduce the skewness and makes it easier to analyze and interpret the data, especially when dealing with outliers or extreme values in the 'price' column.

Sorting the column neighborhood_cleansed by their median price

To sort the 'neighborhood_cleansed' column by their median price, we first calculated the number of listing, the median price and review_scores_rating for each neighborhood using the .groupby() method in Pandas. Then, we sorted the neighborhoods based on their median prices in descending order. with this method we find out The top three neighborhoods with the highest median price are: 1. Haselhorst with a median price of 236.5 and average rating of 4.86 2. West 5 with a median price of 203 and average rating of 4.79 3. Schmargendorf with a median price of 172 and average rating of 4.66

Calculating a trimmed mean

To calculate a more reliable average price, we defined a custom trimmed_mean function that removes the top and bottom 5% of the price data before calculating the mean.

```
def trimmed_mean(x, pct):
    x = sorted(x)
    n = int(len(x) * (pct / 100))
    for i in range(n):
        del x[0]
        del x[-1]
    return sum(x) / len(x)
```

This was done to eliminate the effect of extreme outliers — such as unusually cheap or suspiciously expensive listings — that could distort the result. The function first sorts the prices, then removes the 5% lowest and 5% highest values using a loop, and finally returns the mean of the remaining 90% of the data. We then compared the trimmed mean to the regular arithmetic mean by dividing them. The results showed:

Metric	Value
Trimmed Mean	119.55
Arithmetic Mean	201.24
Comparison (Ratio)	0.59

Since the trimmed mean is lower than the arithmetic mean, this indicates that high-priced outliers are pulling the regular average upward. This means the trimmed mean gives a more realistic picture of what a typical Berlin Airbnb listing actually costs, as it is not influenced by a small number of extremely expensive listings in the dataset.

diagrams to explore the relationship between price and other variables

we actually created several diagrams to explore the relationship between price and other variables, such as 'room_type', 'host_is_superhost', and 'review_scores_rating'. you can see the first to diagrams below:

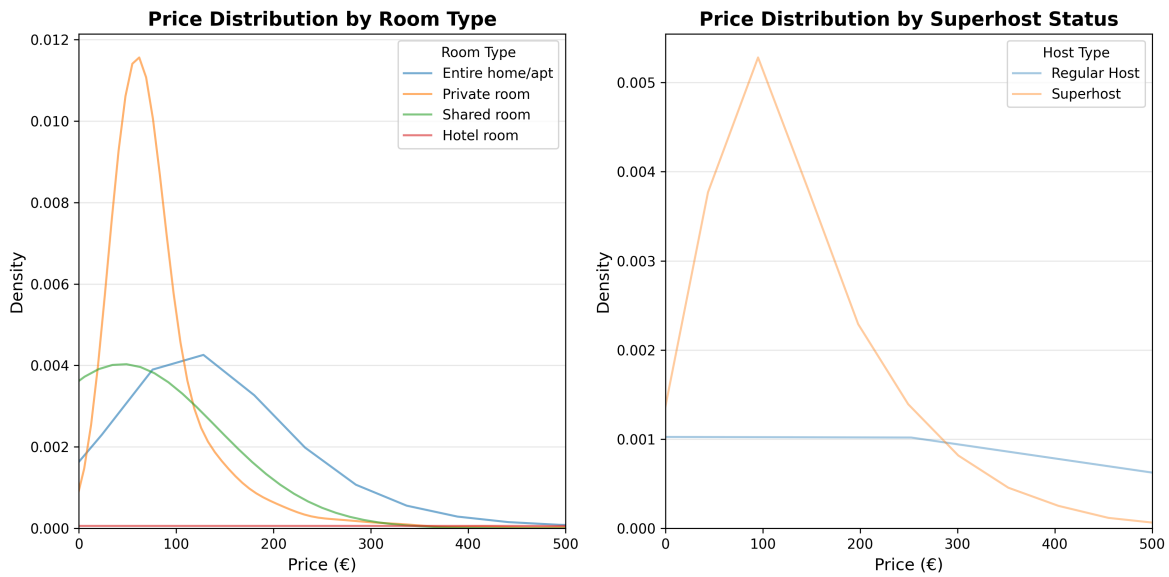

Figure 2: price_by_host_status.png

interpreting the rooms type diagram

As shown in the first plot, the price distribution varies by room type. Entire homes or apartments tend to have higher prices compared to private rooms and shared rooms, which is expected since they offer more privacy and amenities. However, this diagram does not accurately represent the real distribution of hotel room prices.

When the number of hotel rooms and their distribution were examined more closely, it was found that the number of hotel rooms is very similar to the number of shared rooms (110 and 120, respectively). However, the standard deviation of hotel room prices is much higher than that of shared rooms (14,667 and 232, respectively). This indicates that the price distribution for hotel rooms is much more spread out and shows greater variability compared to shared rooms. As a result, the diagram displays the hotel room distribution as a line with very low density.

interpreting the superhost status diagram

The price distribution for both host types peaks around €50-100. However, regular hosts show a higher density at lower prices, meaning their listings are more concentrated in the budget range. Superhosts show a flatter and wider distribution, suggesting they offer listings across a broader range of price points. This may indicate that superhost status is not strictly associated with higher pricing, but rather with more diverse listing types.

and here comes the third diagram that was created in the shape of a scatter plot to explore the relationship between price and review_scores_rating:

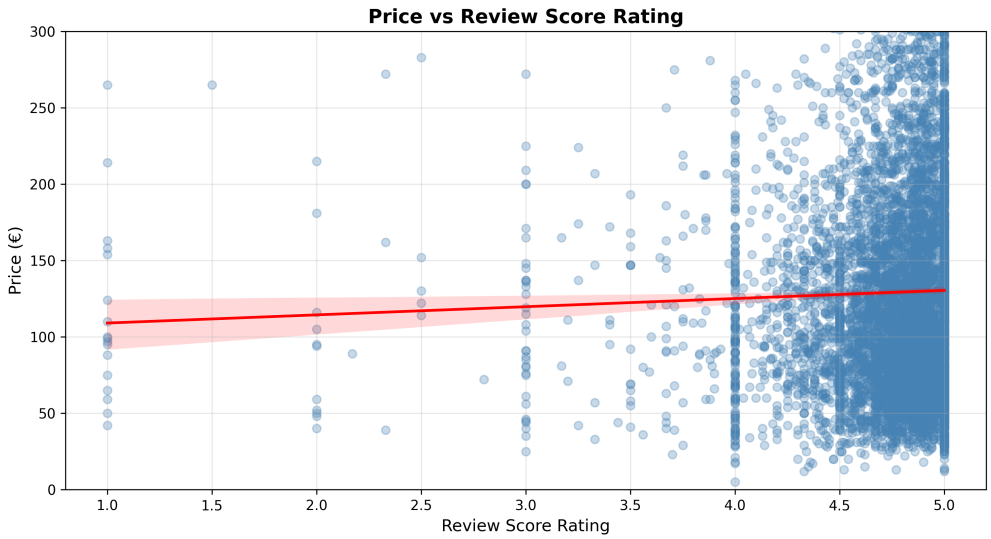

Figure 3: price_vs_review_score.png

interpreting the price vs review score rating diagram

The scatter plot shows the relationship between review score rating and price. The regression line indicates a slight positive trend — listings with higher ratings tend to charge marginally higher prices. However, the relationship is very weak, suggesting that review scores alone do not determine pricing. The majority of listings are clustered between ratings 4.0 and 5.0, with prices varying widely in that range. This indicates that other factors such as location, room type, or number of amenities likely play a much stronger role in determining the price of a listing than the review score alone.

loading GeoJSON file and preparing it for visualization

To create the map visualization, we first needed to match the neighbourhood names in the Airbnb dataset with the official Berlin neighbourhood boundaries from a GeoJSON file. This

required several steps because the neighbourhood names were not written the same way in both datasets.

First, the GeoJSON file was loaded and all official Berlin neighbourhood names were extracted into a list. Then we compared these names with the neighbourhood names in the Airbnb dataset. The initial comparison showed that many names did not match directly because of differences in spelling, special German characters such as ß and ü, extra spaces, and other formatting inconsistencies. To fix this, a **normalize function** was created to standardize the names in both datasets. This function converted all names to lowercase, removed extra spaces and special characters, and replaced common variations such as strasse with straße. After applying this normalization, the number of matched neighbourhoods increased significantly.

However, some neighbourhoods still remained unmatched. For these cases, we **manually created a mapping dictionary** to link the remaining unmatched names in the Airbnb dataset to their corresponding official names in the GeoJSON file. This final step allowed us to successfully match all neighbourhoods and prepare the data for visualization on the map.

there was actually some method to automatically match the names using fuzzy string matching techniques, but due to the complexity and the number of unmatched names, we found that a combination of normalization and manual mapping was more efficient and accurate for our specific dataset.

Finally, the Airbnb listings were merged with the GeoJSON neighbourhood data using the normalized neighbourhood names as the common key. This produced a combined dataset in which each Airbnb listing is linked to its official Berlin neighbourhood boundary, making the data ready for accurate map visualization.

visualizing Airbnb prices across Berlin using a choropleth map

To visualize Airbnb prices across Berlin, we created an interactive choropleth map showing the median nightly price for each neighbourhood. First, we grouped the merged dataset by neighbourhood and calculated the total number of listings and the median price for each area. Since the normalization process had replaced the original names, we merged them back into the dataframe so they could be displayed correctly on the map.

To link the dataframe with the GeoJSON boundaries, we added a match_key property containing the normalized name to each neighbourhood in the GeoJSON file. This allowed Plotly to correctly match each row in the dataframe with its corresponding shape on the map.

The resulting map uses a green-to-blue color scale, where darker colors represent higher median prices. When hovering over a neighbourhood, its name and median price are displayed, making it easy to quickly identify the most and least expensive areas in Berlin. The map also includes a legend to interpret the color scale and a title for context. This visualization provides a clear and intuitive way to explore the spatial distribution of Airbnb prices across the city. you can see the map below:

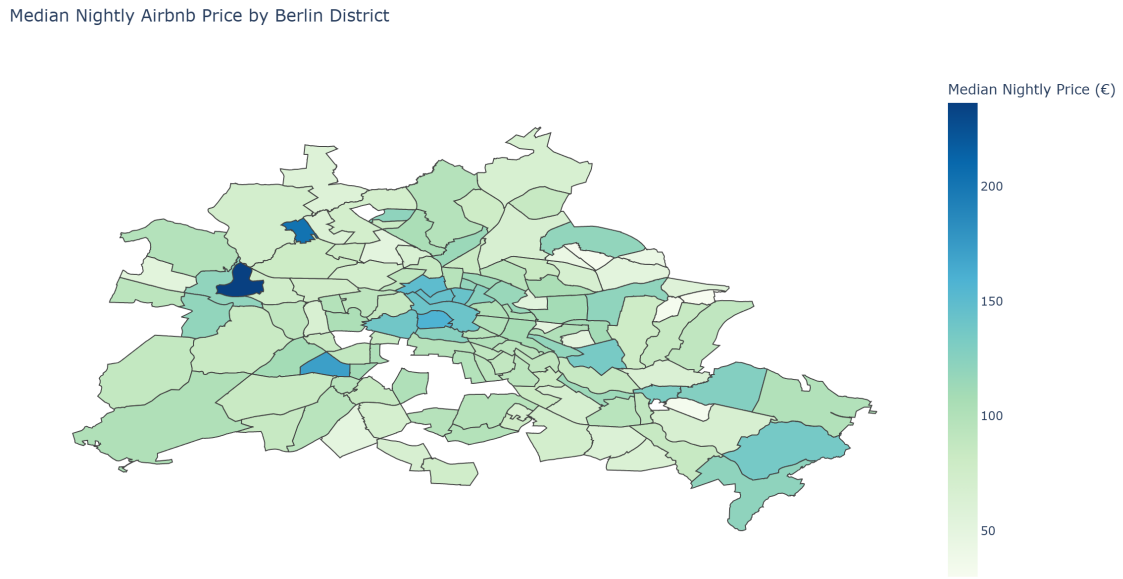

Figure 4: Median Nightly Airbnb Price by Berlin District

Visualizing Individual Airbnb Listings Relative to District Median Prices

To make the above choropleth map more informative, we added another layer that shows each Airbnb listing as a point on the map. Each point is colored depending on whether the listing's price is above or below the median price of its neighbourhood.

First, we used **.transform('median')** to calculate the median price for each neighbourhood and stored it in a new column called median_price_bzr. Then, we compared the price of each listing with this value and **created a new column, price_vs_median, where each listing is labeled as 'Above median' or 'Below median'**.

Next, we created a scatter geo plot with Plotly, where each listing is placed according to its coordinates. **Listings above the median are shown in yellow**, while those **below the median are shown in purple**. This layer was then added on top of the choropleth map from before.

In the end, the map has two layers: the background shows the median price for each neighbourhood, and the points show how individual listings compare to that median. This makes it easier to see both the overall price patterns and the variation within each neighbourhood. thw resulting map allows users to quickly identify which listings are more expensive or cheaper

than the average for their area, providing deeper insights into the pricing landscape of Berlin's Airbnb market. you can see the updated map below:

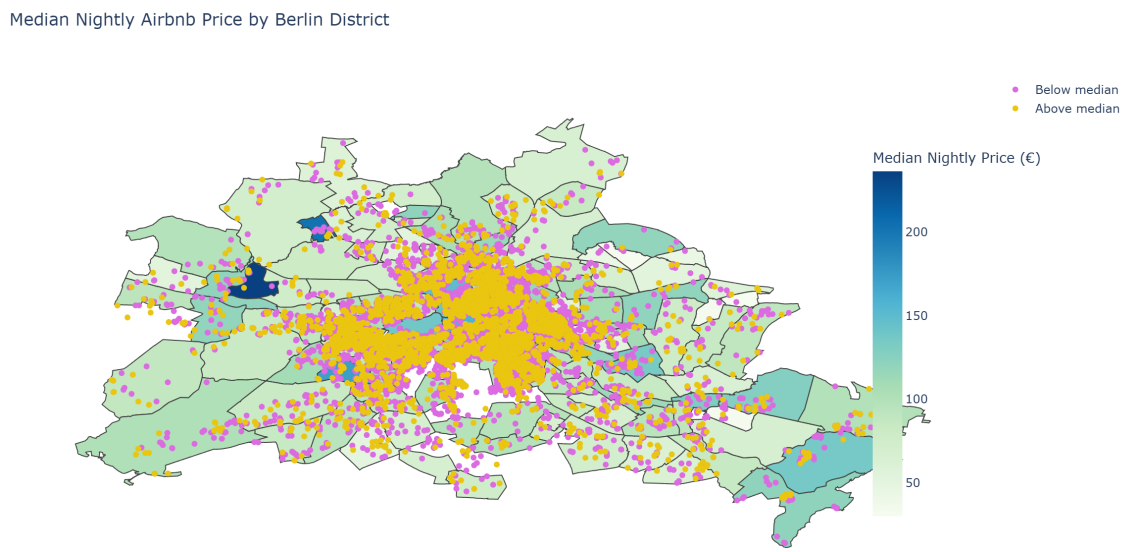

Figure 5: price_vs_median.png

Visualizing Airbnb Listing Density Across Berlin

To complement the first Berlin price map , we created a second choropleth map that shows the listing density, meaning the number of Airbnb listings per km², for each Bezirksregion.

To calculate this, we took the area in m² (GROESSE_m2) from the GeoJSON file for each district and stored it in a separate dataframe called areas_df. This was then merged with the existing group_by_bzr dataframe. Finally, the density was calculated by dividing the number of listings by the area converted from m² to km². you can see the core codes for this map below:

```
areas_df = pd.DataFrame(areas)
group_by_bzr = group_by_bzr.merge(areas_df, on='neighbourhood_norm', how='left')
group_by_bzr['density'] = (group_by_bzr['num_listings']) / (group_by_bzr['area_m2'] / 1000000
```

The resulting map uses a teal–rose color scale, where darker colors represent higher listing concentrations. When hovering over a district, the number of listings in that area is displayed.

This visualization provides insights into which areas of Berlin have the highest concentration of Airbnb listings, which can be useful for understanding market saturation and identifying potential opportunities for new hosts. you can see the map below:

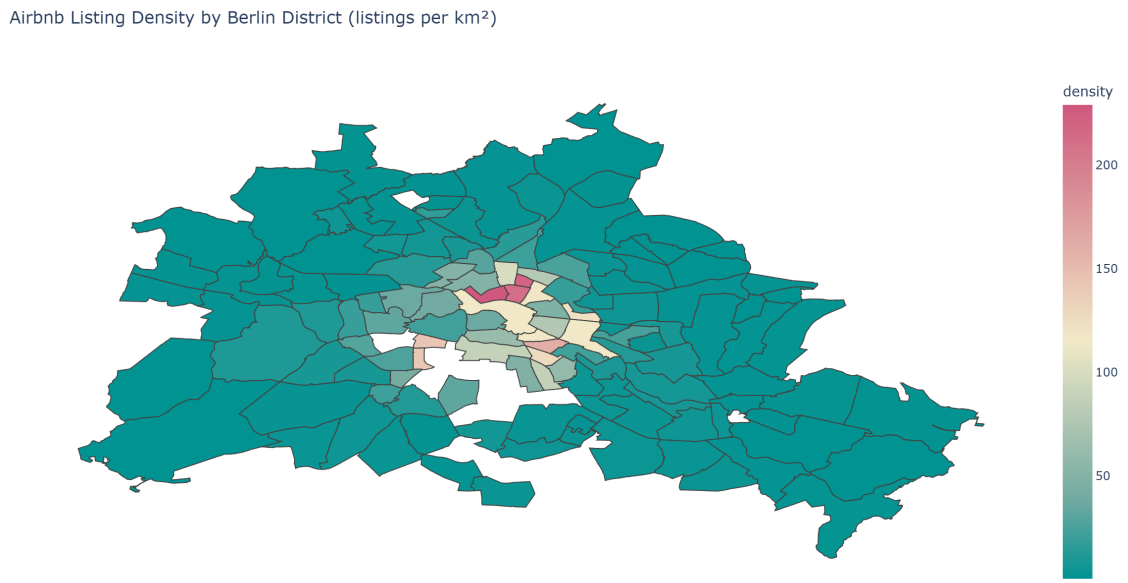

Figure 6: Density Map

interpreting the listing density map and comparing it to the price map

first to gain more clearity we attached Two tables with the top 10 districts with the highest median price and the top 10 districts with the highest listing density:

Table 1

neighbourhood_norm	num_listings	median_price	neighbourhood_cleansed	area_m2	density
brunnenstraße süd	402	145.5	Brunnenstr. Süd	1755172	229.04
helmholtzplatz	157	119.0	Helmholtzplatz	711300	220.72
prenzlauer berg südwest	290	144.0	Prenzlauer Berg Südwest	1370147	211.66
südliche luisenstadt	242	101.0	südliche Luisenstadt	1496159	161.75
schöneberg nordwest	212	85.0	Schöneberg-Nord	1479646	143.28
schöneberg südwest	163	103.0	Schöneberg-Süd	1150672	141.66
reuterstraße	181	90.0	Reuterstraße	1426231	126.91
frankfurter allee nord	156	104.5	Frankfurter Allee Nord	1280543	121.82

neighbourhood_norm	num_listings	median_price	neighbourhood_cleansed	area_m2	density
prenzlauer berg süd	197	125.0	Prenzlauer Berg Süd	1660928	118.61
frankfurter allee süd fk	477	108.0	Frankfurter Allee Süd FK	4090904	116.60

Table 2

neighbourhood_norm	num_listings	median_price	neighbourhood_cleansed	area_m2	density
haselhorst	6	236.5	Haselhorst	4693583	1.28
west 5 tegel	11	203.0	West 5	2392478	4.60
schmargendorf	34	172.0	Schmargendorf	4082997	8.33
regierungsviertel	98	159.0	Regierungsviertel	2695389	36.36
brunnenstraße nord	161	150.0	Brunnenstr. Nord	3320296	48.49
brunnenstraße süd	402	145.5	Brunnenstr. Süd	1755172	229.04
prenzlauer berg südwest	290	144.0	Prenzlauer Berg Südwest	1370147	211.66
alexanderplatz	725	142.0	Alexanderplatz	6228830	116.39
altstadt kietz	5	139.0	Altstadt-Kietz	nan	nan
tiergarten süd	115	139.0	Tiergarten Süd	5171032	22.24

As you can see from the maps and tables, the highest listing densities are found in Brunnenstraße Süd (229 listings/km^2), Helmholtzplatz (221 listings/km^2), and Prenzlauer Berg Südwest (212 listings/km^2). These are all relatively small, centrally located neighbourhoods with strong tourist demand.

When comparing this to the price map from above, the patterns do not fully match. Areas with a high density of listings are not always the most expensive. For example, Brunnenstraße Süd has the highest density but only a moderate median price of €145.5, while Haselhorst has one of the lowest densities but a much higher median price of €236.5. This suggests that listing density is mainly influenced by central location and tourist demand, whereas prices are more affected by factors such as listing quality and specific neighbourhood characteristics.

Section 2

Introduction

Section two looks at what drives Airbnb listing prices in Berlin using regression analysis. Using the cleaned data from Section 1, we estimate and compare OLS, Ridge, and LASSO models to see how property, host, and location features affect nightly prices. The target variable is the log of price, so coefficients show approximate percentage changes. We also check model fit, run diagnostics, and compare prediction accuracy. The goal is to identify the main factors behind pricing and give practical insights for hosts.

cleaning and preparing the data for regression analysis

At first we load and clean the data as we did in section one, we personally preferd to writte the codes for each secion in different Notebooks to avoid any changes to the original data and to make it more clear and organized. after cleaning the price column and making sure there is no None value and 0 in our list, we created a new column called log_price by applying the natural logarithm to the price column. Then We picked seven numeric variables as predictors, like the number of bedrooms, review scores, and availability. For the categorical variables, instant_bookable and room_type, we used one-hot encoding to turn them into dummy variables, leaving out the first category to avoid multicollinearity. Finally, all the columns were converted to float numbers so they could be used in the regression. After preprocessing, we checked the sample size and data types, made sure instant_bookable was correctly encoded as 0 and 1, and confirmed that all columns were numeric. This ensured the dataset was ready to be used for regression.

Estimating the Effect of Listing Characteristics on Airbnb Prices

the next step is to identify which listing characteristics significantly influence Airbnb prices in Berlin but When selecting variables for the regression model, **we encountered many missing values in the review-related variables**. We had two options: either **exclude these variables** or **remove all observations with missing values**. Since the number of missing values was relatively high, we were concerned that dropping them could affect the results. Therefore, we estimated two models.

1. including review variables

Table 3: OLS Regression Results

Dep. Variable:	price	R-squared:	0.447
Model:	OLS	Adj. R-squared:	0.446

Method:	Least Squares	F-statistic:	473.8
Date:	Tue, 17 Mar 2026	Prob (F-statistic):	0.00
Time:	20:00:17	Log-Likelihood:	-4310.5
No. Observations:	6482	AIC:	8643.
Df Residuals:	6471	BIC:	8718.
Df Model:	10		
Covariance Type:	HC3		

	coef	std err	z	P>|z|	[0.025	0.975]
const	3.7831	0.128	29.599	0.000	3.533	4.034
bedrooms	0.1267	0.013	10.098	0.000	0.102	0.151
bathrooms_text	0.0926	0.015	6.060	0.000	0.063	0.123
review_scores_rating	0.0853	0.026	3.327	0.001	0.035	0.136
availability_365	0.0001	5.16e-05	2.840	0.005	4.55e-05	0.000
reviews_per_month	-0.0153	0.004	-4.319	0.000	-0.022	-0.008
accommodates	0.0808	0.005	15.799	0.000	0.071	0.091
instant_bookable_t	0.2344	0.014	16.605	0.000	0.207	0.262
room_type_Hotel room	0.5252	0.123	4.261	0.000	0.284	0.767
room_type_Private room	-0.3694	0.015	-25.012	0.000	-0.398	-0.340
room_type_Shared room	-1.1293	0.093	-12.098	0.000	-1.312	-0.946

Omnibus:	2991.842	Durbin-Watson:	1.596
Prob(Omnibus):	0.000	Jarque-Bera (JB):	70272.178
Skew:	1.677	Prob(JB):	0.00
Kurtosis:	18.778	Cond. No.	3.44e+03

2. excluding review variables

Table 4: OLS Regression Results

Dep. Variable:	price	R-squared:	0.447
Model:	OLS	Adj. R-squared:	0.446
Method:	Least Squares	F-statistic:	473.8
Date:	Tue, 17 Mar 2026	Prob (F-statistic):	0.00
Time:	20:09:48	Log-Likelihood:	-4310.5
No. Observations:	6482	AIC:	8643.
Df Residuals:	6471	BIC:	8718.
Df Model:	10		

Covariance Type: HC3

	coef	std err	z	P>|z|	[0.025	0.975]
const	3.7831	0.128	29.599	0.000	3.533	4.034
bedrooms	0.1267	0.013	10.098	0.000	0.102	0.151
bathrooms_text	0.0926	0.015	6.060	0.000	0.063	0.123
review_scores_rating	0.0853	0.026	3.327	0.001	0.035	0.136
availability_365	0.0001	5.16e-05	2.840	0.005	4.55e-05	0.000
reviews_per_month	-0.0153	0.004	-4.319	0.000	-0.022	-0.008
accommodates	0.0808	0.005	15.799	0.000	0.071	0.091
instant_bookable_t	0.2344	0.014	16.605	0.000	0.207	0.262
room_type_Hotel room	0.5252	0.123	4.261	0.000	0.284	0.767
room_type_Private room	-0.3694	0.015	-25.012	0.000	-0.398	-0.340
room_type_Shared room	-1.1293	0.093	-12.098	0.000	-1.312	-0.946

Omnibus:	2991.842	Durbin-Watson:	1.596
Prob(Omnibus):	0.000	Jarque-Bera (JB):	70272.178
Skew:	1.677	Prob(JB):	0.00
Kurtosis:	18.778	Cond. No.	3.44e+03

and compared their performance using R^2, The model including reviews showed a higher R^2, indicating a better fit. As a result, we chose to include the review variables and removed o bservations with missing values, even though this reduced the sample size by about one-third.

Results Interpretation

The results show that hotel rooms, instant-bookable listings, and the number of bedrooms have the strongest positive effects on Airbnb prices in Berlin. For example, hotel room listings are priced about 0.53 log units higher than entire homes or apartments on average. Instant-bookable listings are around 23.4% more expensive, holding other factors constant. In addition, each extra bedroom increases the price by about 12.6%.

On the other hand, shared rooms have a strongly negative effect on price, meaning they are much cheaper due to lower privacy. The R^2 value indicates that the model explains about 44.7% of the variation in Airbnb prices. This suggests that while the included variables are important, there are still other factors not captured by the model that influence pricing, such as location, amenities, or seasonal demand.

Evaluating Predictive Performance through Model Diagnostics

To evaluate the performance of the OLS model, we created two diagnostic plots. The first plot shows residuals against fitted values to check if the main assumptions of linear regression are met. The second plot compares the actual log prices with the predicted values to see how well the model performs overall.

Residuals vs Fitted Values

In the residuals vs fitted values plot, we look for patterns in the residuals. Ideally, the residuals should be randomly scattered around zero without any clear pattern. but as you can see below in the plot, the residual plot shows a funnel shape, with more spread in the middle and less at higher fitted values, indicating heteroscedasticity (non-constant variance). This supports the use of HC3 robust standard errors to handle this issue. There are also a few outliers with large positive residuals, meaning some listings are priced much higher than the model predicts and may affect the results.

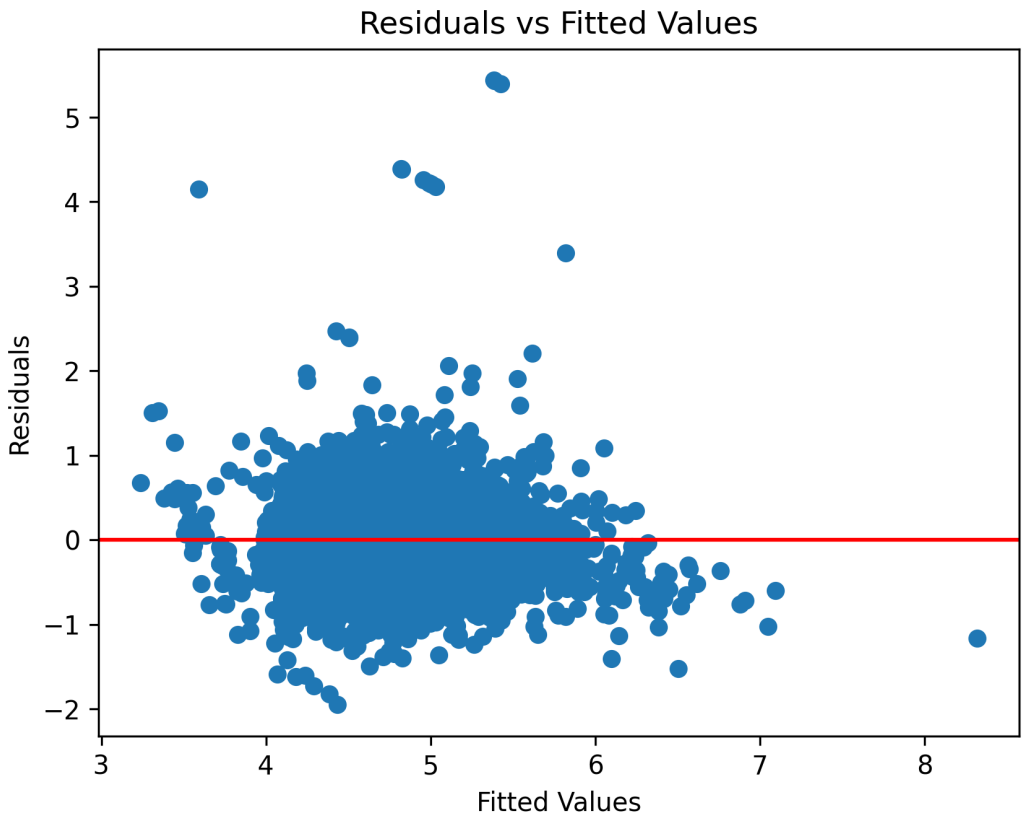

Figure 7: Residuals vs Fitted Values

True vs Fitted Values

In the true vs fitted values plot, we want to see points clustered around the 45-degree line, which would indicate good predictions and we can see that in the plot below. the red line shows perfect prediction. Most points are close to the line in the lower price range (3–6), meaning the model works well for typical listings. However, for higher prices, the points move away from the line, showing that the model underpredicts expensive listings. This suggests it struggles to capture premium properties.

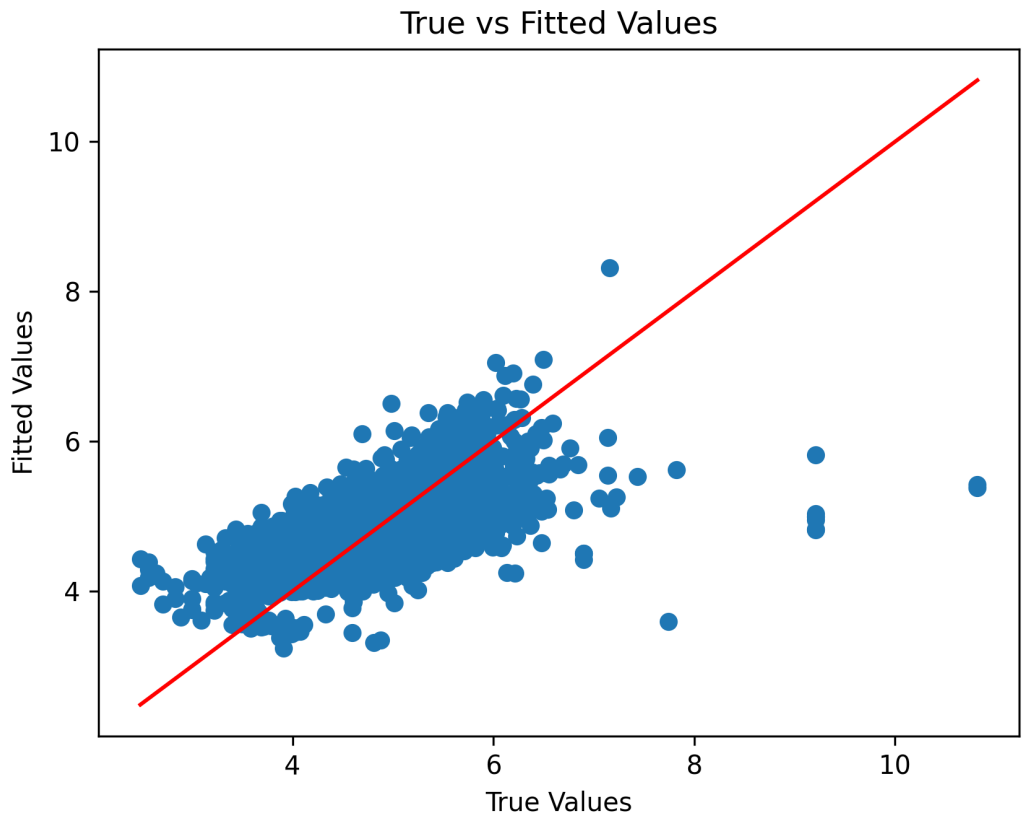

Figure 8: True vs Fitted Values

In short, the main issue is heteroscedasticity. This is why using cov_type='HC3' was a good choice, as it adjusts for this problem and keeps the standard errors and p-values reliable. The model performs reasonably well for average listings but has trouble with outliers and high-priced properties, which is common in real estate data.

Ridge Regression Estimation

To reduce potential instability in the OLS estimates, Ridge regression formula:

$$\hat{\beta}_{Ridge} = (X^\top X + \lambda I)^{-1} X^\top y$$

was applied using its closed-form formula. The function ridge_estimate(X, y, lam) was implemented in NumPy to calculate the coefficients directly.

Here, λ is the regularization parameter and I is the identity matrix. A custom function ridge_estimate(X, y, lam) was implemented in NumPy to compute the Ridge coefficients directly based on this formula. The model was estimated for three values of $\lambda \in 0.01, 1, 100$. For each case, the ℓ_2-norm of the coefficient vector was calculated as:

$$\|\hat{\beta}\|_2 = \sqrt{\sum_{j=1}^{p} \hat{\beta}_j^2}$$

this norm measures the overall magnitude of the coefficients. As λ increases, the Ridge penalty shrinks the coefficients towards zero, which reduces their ℓ_2-norm. here are the results for the coefficients and their ℓ_2-norms for each λ value:

Table 5

Lambda	L2 Norm
0.010000	1.477200
1.000000	1.466600
100.000000	1.067600

The results above shows that as λ increased, the coefficients became smaller and more stable, demonstrating the regularization effect of Ridge regression in controlling for multicollinearity and improving model generalization.

LassoCV Regression — Selecting the Best Alpha and Features

In this task we applied LassoCV regression to automatically find the best regularization parameter (alpha) and eliminate irrelevant features. We set up the model with 10-fold cross-validation and fitted it on our feature matrix X and target variable Y. LassoCV tests multiple alpha values and selects the one with the lowest average error across all folds, saving us from manually tuning the model. After fitting, we plotted the Mean Square Error across different alpha values to visualize how the model performs. as you see below in these plot the best alpha is marked by a vertical dashed line on the plot

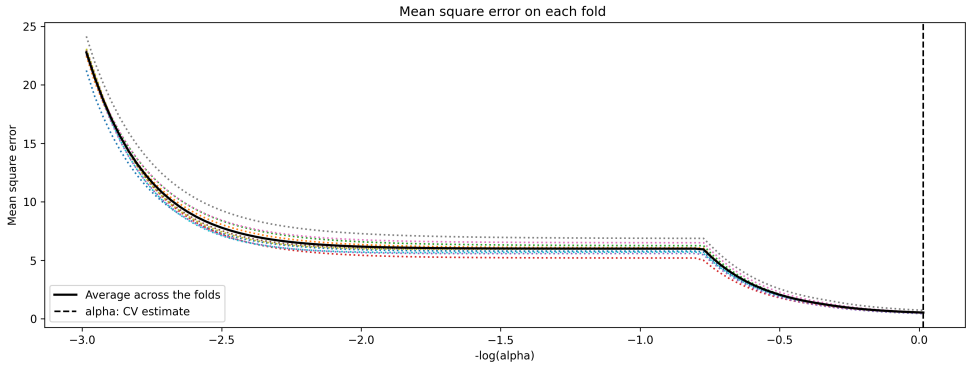

Figure 9: lasso_cv_plot.png

The plot shows how the MSE decreases as alpha increases across all 10 folds. The MSE starts high at around 22-23 on the left and gradually drops to its lowest point at the dashed vertical line, which marks the best alpha selected by LassoCV. This confirms that the model successfully found the optimal alpha where the prediction error is minimized.

Finally, we printed all features whose coefficients were reduced to exactly zero by the model, meaning Lasso decided they had no useful contribution and removed them completely from the model. The features that were eliminated are:

Table 6

Feature	Coefficient	Status
bedrooms	0.000000	Eliminated by Lasso
bathrooms_text	0.000000	Eliminated by Lasso
reviews_per_month	0.000000	Eliminated by Lasso
instant_bookable_t	0.000000	Eliminated by Lasso
room_type_Hotel room	0.000000	Eliminated by Lasso
room_type_Private room	-0.000000	Eliminated by Lasso
room_type_Shared room	0.000000	Eliminated by Lasso

Out-of-Sample Model Comparison (OLS vs Ridge vs Lasso)

In this task we compared three regression models by evaluating their performance on unseen data. We started by splitting the dataset into 80% training and 20% test sets using the train_test_split function with a fixed random state of 77 to ensure the results are reproducible.

We then fitted all three models on the training data and evaluated them on the test set by calculating the RMSE (Root Mean Square Error) for each one. RMSE measures how far the

predictions are from the actual values on average, where a lower RMSE indicates a better performing model.

For the OLS model, we added a constant to both the training and test sets and fitted the model using HC3 robust standard errors to account for heteroscedasticity. For Ridge and Lasso, we used 10-fold cross-validation to automatically find the best alpha, the same approach we applied in the previous task. All three models were then used to predict the target variable on the test set and their RMSE values were printed for comparison.

interpreting the results of the out-of-sample model comparison

Looking at the results, OLS and Ridge performed almost identically with an RMSE of around 0.45, meaning both models predict the target variable with similar accuracy. Ridge performed slightly better, but the difference was very small. Lasso on the other hand had a noticeably higher RMSE of around 0.69, which is likely because it eliminated some features completely, reducing the model's ability to capture all relevant information in the data. Overall, Ridge performed best out of the three models, OLS came in as a close second, while Lasso performed worst in terms of out-of-sample prediction accuracy, likely due to its aggressive feature selection.

summarizing the result and advising a new Airbnb host in Berlin

The regression results show that Airbnb prices in Berlin are mainly influenced by the size of the listing, room type, booking options, and reviews. Larger listings, measured by the number of bedrooms and how many guests they accommodate, tend to have higher prices. Room type is especially important: private and shared rooms are much cheaper than entire apartments, with shared rooms having the lowest prices. Listings that offer instant booking can charge more, likely because guests value convenience. Higher review scores also increase prices, while listings with many reviews per month tend to be slightly cheaper. For a new host, it is important to compare similar listings in the same area. Offering an entire apartment, enabling instant booking, and maintaining good reviews can justify higher prices. Starting with a lower price to gain reviews and then increasing it over time is a good strategy.

LAD Regression (Least Absolute Deviations)

after Sample comparison, we re-estimated the model using LAD regression (Least Absolute Deviations), which minimizes absolute errors instead of squared errors. This makes it more robust to outliers, as large deviations have less influence on the results.

LAD regression can be implemented using Quantile Regression with the quantile set to $q = 0.5$, which corresponds to the median. We used the QuantReg function with the target variable Y and the feature matrix X_c, where X_c includes a constant term. By estimating the model at $q = 0.5$, we examine how the predictors affect the median of Y, rather than the mean as in OLS.

The model output provides coefficient estimates that can be compared with the OLS results. Overall, LAD produces more stable and reliable estimates in the presence of outliers, which are common in price data. you can see the results of the LAD regression below:

Table 7

Feature	Coefficient	Std Error	T-value	P-value	CI 2.5%	CI 97.5%
const	3.663000	0.091000	40.378000	0.000000	3.485000	3.841000
bedrooms	0.139300	0.011000	12.532000	0.000000	0.118000	0.161000
bathrooms_text	0.079000	0.015000	5.127000	0.000000	0.049000	0.109000
review_scores_rating	0.116400	0.018000	6.385000	0.000000	0.081000	0.152000
availability_365	0.000200	0.000055	2.949000	0.003000	0.000054	0.000000
reviews_per_month	-0.016300	0.003000	-5.675000	0.000000	-0.022000	-0.011000
accommodates	0.075000	0.004000	16.979000	0.000000	0.066000	0.084000
instant_bookable_t	0.196800	0.014000	13.734000	0.000000	0.169000	0.225000
room_type_Hotel room	0.421300	0.058000	7.324000	0.000000	0.309000	0.534000
room_type_Private room	-0.408200	0.016000	-25.863000	0.000000	-0.439000	-0.377000
room_type_Shared room	-1.093800	0.058000	-18.729000	0.000000	-1.208000	-0.979000

interpreting the results of the LAD regression

Comparing the LAD and OLS results, both models produce very similar coefficients, suggesting that the relationships between the features and the target variable are quite robust. The biggest differences appear for variables like review scores, hotel room type, and instant booking, which indicates that these may be slightly affected by outliers.

However, the signs of all coefficients remain the same and all variables stay statistically significant, meaning the overall conclusions do not change. This suggests that the OLS results are reliable and not strongly driven by extreme observations.

For this task, we focused on the coefficient of the variable instant_bookable from the model in Task 2, as it is interesting to examine whether instant booking has a significant effect on Airbnb prices in Berlin.

a. Hypotheses The null and alternative hypotheses are defined as:

$H_0 : \beta_{instant_bookable} = 0$ $H_1 : \beta_{instant_bookable} \neq 0$

b. HC3-Robust Test Statistic and P-value Using HC3-robust standard errors, the test statistic is $z = 16.605$ with a p-value of $p = 0.000$. Since the p-value is far below common significance levels, we reject the null hypothesis and conclude that instant booking has a statistically significant effect on price.

c. 95% Confidence Interval The 95% confidence interval based on HC3 standard errors is:

$[0.207,\ 0.262]$

Since zero is not included in this interval, this further supports rejecting the null hypothesis and confirms that the effect is statistically significant.

d. Real-World Interpretation

The estimated coefficient is:

$\hat{\beta}_{instant_bookable} = 0.2344$

Because the dependent variable is $\log(price)$, we convert the coefficient as follows:

$\exp(\hat{\beta}) - 1 = \exp(0.2344) - 1 \approx 0.264$

This means that instant bookable listings are associated with prices that are about 26.4% higher compared to non-instant bookable listings, holding all other factors constant. In the Berlin Airbnb market, this suggests that the convenience of instant booking allows hosts to charge a noticeable price premium.

conclusion about Heteroscedasticity, Causality, and Omitted Variable Bias

to wrap things up, we need to explain some points about heteroscedasticity, causality, and omitted variable bias in the context of our regression analysis.

Heteroscedasticity

Heteroscedasticity-robust standard errors (HC3) are important because the variance of the errors is unlikely to be constant across all listings. For example, more expensive or luxury properties often show greater price variation than cheaper ones. If heteroscedasticity is ignored, the standard errors, test statistics, and p-values may become unreliable. Using HC3 helps correct for this issue and ensures that our inference remains valid.

Causality and Omitted Variable Bias

When it comes to causality, the coefficient for instant_bookable should not be interpreted as a causal effect, but only as an association. There are likely omitted variables—such as listing quality, host experience, neighborhood attractiveness, and available amenities—that influence both the decision to enable instant booking and the listing price. This creates bias in the estimate and means we cannot draw causal conclusions from the model.

Section 3

Introduction

Section three focuses on building and evaluating classification models to predict credit default risk. The dataset consists of 5000 borrowers described by 20 financial and demographic features — such as income, credit score, debt-to-income ratio, and payment history — along with a binary label indicating whether a borrower defaulted. **The goal of this section is to develop and compare multiple classification approaches, including logistic regression, k-nearest neighbors, support vector machines, and random forests, in order to identify the most reliable model for credit scoring and apply it to new, unseen customers.**

Loading and Exploring the Credit Default Dataset

first we load dataset like what we did in section one, using the URL for the CVS file in our Github repository. after loading the data, we reported the dataset dimensions, computed the fraction of defaulted borrowers, and generated a summary statistics table covering the mean, standard deviation, minimum, and maximum for all 20 features. you can see the results below:

annual_income	age	employment_years	debt_to_income	credit_score	credit_utilization	nu
5000.000000	5000.000000	5000.000000	5000.000000	5000.000000	5000.000000	50
50309.452058	43.626200	5.733020	0.166050	678.794200	0.280992	4.9
29349.199007	12.364216	4.166518	0.159139	75.449405	0.156996	2.1
5473.630000	22.000000	0.100000	0.007300	434.000000	0.006200	0.0
30072.305000	33.000000	2.900000	0.072300	626.000000	0.161350	3.0
44019.125000	44.000000	4.600000	0.118250	681.000000	0.259400	5.0
62777.470000	54.000000	7.325000	0.201425	732.000000	0.384725	6.0
338861.400000	65.000000	40.000000	1.500000	850.000000	0.869900	13

Now to answer the questions regarding the **scaling**, we can explain that Since distance-based methods like KNN depend heavily on how features are scaled, we standardized all variables using StandardScaler from sklearn. This way, features with larger values, such as annual_income, don't dominate the distance calculations compared to smaller-scale features like num_dependents.

Logistic Regression

at first we gave a brief introduction about logistic regression and why we standardized the features before fitting the model. Logistic regression is a linear model used for binary classification. It estimates the probability that a given input belongs to the default class (1) based on a linear combination of the features. The logistic function maps any real-valued number into the (0, 1) interval, making it suitable for modeling probabilities. In the codes we standardized all features using StandardScaler so that the coefficients can be compared more fairly across variables. gere is a summery of the standardization codes:

```
# standardizing features to mean 0 and standard deviation 1
scaled_X_train = X_train.copy()
scaler = StandardScaler()
scaled_X_train[X_train.columns] = scaler.fit_transform(scaled_X_train[X_train.columns])
scaled_X_train.head()
```

Without this step, features with larger values, like annual_income, might seem more important than smaller-scale ones just because of their size, not because they actually have more predictive value.

After that, we trained a logistic regression model and looked at the three features with the highest absolute coefficient values: ['previous_defaults', 'credit_utilization', 'debt_to_income']. These features have the biggest impact on predicting whether a default will occur.This makes intuitive sense — a borrower who has defaulted before is clearly at higher risk, someone using most of their available credit is likely financially stretched, and a high debt-to-income ratio shows that a big part of their income is already going toward paying off debt. The model reached a training accuracy of 72.98%.

K-Nearest Neighbors

Next, We ran KNN using three values of k (5, 20, 50) and two weighting methods (uniform and distance), evaluating each setup with 5-fold cross-validation. Cross-validation gives a more reliable measure of accuracy than a single train-test split because it tests the model on multiple subsets of the data.

The best result came from k=20 with distance weighting, which achieved an accuracy of 71.92%. Overall, larger k values worked better than k=5, showing that considering more neighbors helps reduce noise. The difference between uniform and distance weighting was small, though distance weighting usually performed slightly better. you can see the results for all KNN configurations below:

Table 9

k	weights	Accuracy
5	uniform	0.687400
5	distance	0.687400
20	uniform	0.713200
20	distance	0.719200
50	uniform	0.710800
50	distance	0.713000

KNN might struggle with this dataset because it has 20 features. In high-dimensional spaces, distances between points become less meaningful since everything starts to seem similarly far apart — a problem known as the curse of dimensionality.

Support Vector Machine

We also trained two SVM models — one with a linear kernel and one with an RBF kernel using 5-fold cross-validation.

The linear SVM achieved an accuracy of 71%, which is similar to logistic regression. This makes sense because both methods try to separate borrowers using a straight line.

For the RBF kernel, we tested three values of C (0.1, 1, 10). The parameter C controls how strict the model is — a higher C makes the model try harder to classify every point correctly, which can lead to overfitting. The best result was C=1, giving an accuracy of 74.34%. When C was increased to 10, accuracy dropped, showing that the model became too strict and lost its ability to generalize.

Overall, the RBF kernel performed better than the linear one, indicating that the boundary between defaulters and non-defaulters is not a simple straight line but has a more complex shape.

Decision Boundary Visualization

We selected previous_defaults and credit_utilization and trained both models using just these two features, then plotted their decision boundaries side by side. you can see the resulting plot below:

Decision Boundary Plot

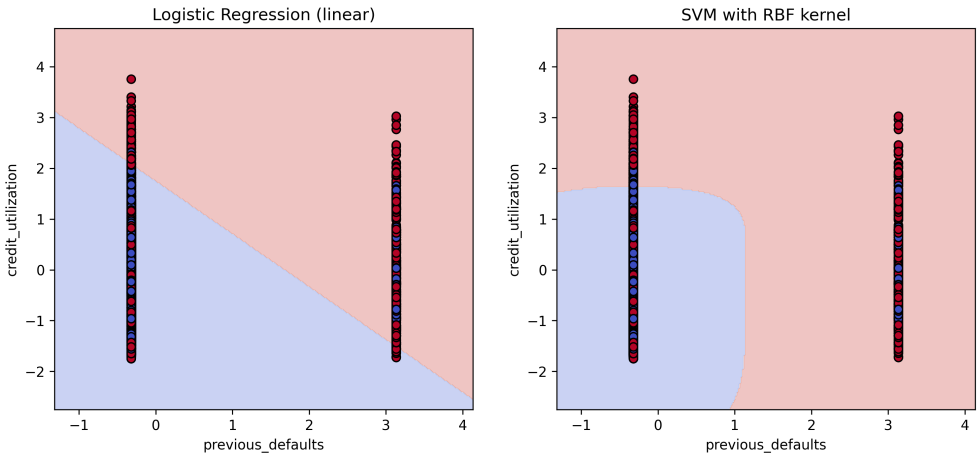

Figure 10: comparison_plot.png

as you can see, in the logistic regression plot, the boundary is a straight diagonal line, simply splitting the space into two halves. In the SVM RBF plot, the boundary curves around the data points, allowing a more flexible separation between defaulters and non-defaulters.

Looking at the data, most borrowers cluster in two vertical strips (previous_defaults = 0 and 1), which makes classification tricky with only two features. The SVM handles this better by bending its boundary around the clusters, while logistic regression can only make a straight cut. This explains why the RBF kernel achieved higher accuracy in previous experiments.

Random Forest

finally we trained a random forest by testing different combinations of n_estimators (50, 100, 200) and max_depth (3, 5, 10) using 5-fold cross-validation. The best setup was n_estimators = 50 and max_depth = 10, achieving an accuracy of 74.12%.

The top 5 most important features were credit_utilization, debt_to_income, annual_income, credit_score, and payment_history_score. see the feature importance table below:

Table 10

feature	importance
credit_utilization	0.184857
debt_to_income	0.147405

feature	importance
annual_income	0.088723
credit_score	0.068666
payment_history_score	0.065013

Compared to logistic regression, there are both similarities and differences. credit_utilization and debt_to_income appear important in both models, confirming they are strong predictors of default. However, previous_defaults, which was the most important feature in logistic regression, does not appear in the random forest's top 5. This happens because random forests distribute importance across many trees and features, while logistic regression assigns a single large coefficient. On the other hand, annual_income and credit_score are new in the top 5, showing that the random forest can capture patterns that logistic regression might miss.

Model Comparison

as the final step, we compared the performance of all models on the test set by calculating their accuracy. The results were as follows:

Model	Accuracy
Logistic Regression	72.98%
KNN (k=20, distance)	71.92%
SVM RBF (C=1)	74.34%
Random Forest (n=50, depth=10)	74.12%

The best-performing model was the SVM with an RBF kernel, with Random Forest coming in a close second. KNN performed the worst, probably because the high number of features makes distance calculations less reliable.

However, accuracy might not be the most useful metric here. As we saw in Task 1, the dataset is imbalanced — there are more borrowers who did not default than those who did. In this situation, a model could get a decent accuracy just by predicting "no default" for everyone, without actually learning meaningful patterns for identifying real defaulters.

Predicting Default Probabilities for New Customers

to finalize our analysis, we applied our best model from previous work — the SVM with RBF kernel (C=1) — to predict default probabilities for 2,500 new customers. We first standardized the test data using the same scaler fitted on the training set to keep the data consistent.

By setting **probability=True** in the SVM, the model outputs probabilities **instead of just a yes/no** prediction. These probabilities show how likely each customer is to default — for example, a probability of 0.65 means the model estimates a 65% chance that the borrower will default.

The first five predictions show a mix of low and high probabilities, indicating the model is effectively distinguishing between higher-risk and lower-risk borrowers. The results were saved to predictions.csv, including each customer's ID and their predicted default probability. This allows lenders to prioritize which customers to review more closely based on their risk level, improving decision-making in credit approval processes. the CVS file results can be found in our Github repository.

The Last Word

Thank you for your attention, We know we nailed it.

Declaration of Authenticity

"We/I hereby certify that We/I have independently prepared the foregoing paper without external assistance and have not used any aids other than those indicated in the attached list. All passages taken verbatim or in substance from publications have been marked as such. Furthermore, We/I certify that We/I have not previously submitted this paper in any other examination procedure."

Place, Date, Signature (1) _____

Place, Date, Signature (2) _____

Place, Date, Signature (3) _____

Place, Date, Signature (4) _____